

Algoritmos y Estructuras de Datos
Examen Final 29/07/2021 – Regulares 2015+ [Python y PE]

Una compañía de catering (servicios de comidas para eventos) necesita un programa que le permita gestionar los datos de los servicios que tiene en su catálogo. De cada **Servicio**, se tiene su *número de identificación* (un entero), su nombre (una cadena), la *descripción del servicio prestado* (una cadena con un texto terminado en punto y con palabras separadas por un blanco (por ejemplo: "Almuerzo para hasta 100 personas."), el *precio del servicio* (un valor de tipo float), un número entero entre 0 y 9 que indica el tipo de servicio (por ejemplo: 0: para empresas, 2: para familias, 3: para egresados, etc.) y otro número entero pero entre 1 y 5 que indica la forma de pago que se acepta para ese servicio (por ejemplo, 1: efectivo, 2: tarjeta de crédito, etc.).

En base a lo anterior, desarrollar un programa completo que disponga al menos de dos módulos:

- En uno de ellos, definir la clase **Servicio** que represente al registro a usar en el programa, y las funciones básicas para operar con registros de ese tipo. Este módulo deberá contener también la definición de cualquier otro tipo de registro que se indique.
 - En otro módulo, incluir el programa principal y las funciones generales que sean necesarias. Para la carga de datos, aplique las validaciones que considere necesarias. El programa debe basarse en un menú de opciones para desarrollar las siguientes tareas:
1. Generar un **arreglo de registros** que contenga los datos de todos los productos. Puede generarlo cargando los datos en forma manual o generando los datos en forma aleatoria. No se exige que el arreglo permanezca ordenado durante el proceso de carga, ni tampoco que se ordene al finalizar la carga, pero debe considerar que esta opción puede ser invocada varias veces a lo largo del programa, y que en cada ejecución pueden agregarse tantos registros como desee el operador, sin eliminar los datos que ya estaban cargados. Será considerada la eficiencia de la estrategia de carga y los algoritmos que aplique.
 2. Mostrar el arreglo generado en el punto anterior, a razón de un registro por línea en la consola de salida, pero ordenado por número de identificación de servicios.
 3. Mostrar los datos de los servicios cuya forma de pago sea 1, a razón de un registro por línea en la consola de salida, pero de forma que ese listado salga ordenado alfabéticamente por los nombres de los servicios.
 4. Genere a partir del arreglo un **archivo binario de registros**, con los datos de todos los servicios cuyo tipo no sea ni 0 ni 1. Muestre el archivo, y al final del listado agregue una línea adicional indicando el promedio de los precios de los servicios mostrados. El archivo debe crearse desde cero cada vez que se seleccione esta opción.
 5. Determine si existe en el arreglo creado en el punto 1, un servicio cuyo nombre coincida con el valor **nom** que se carga por teclado. Si existe, muestre sus datos completos, detenga la búsqueda y retorne la cadena contenida en el campo *descripción del servicio*. Si no existe, informe con un mensaje y retorne la cadena "No existe."
 6. A partir del arreglo creado en el punto 1, determine cuántos servicios hay para cada combinación posible entre los tipos de servicio y las formas de pago (un total de $10 \times 5 = 50$ contadores). Genere **todos los contadores posibles**, pero muestre solo los resultados que **correspondan a los tipos menores que 6 y que hayan tenido un conteo diferente de cero**.
 7. Tome la cadena retornada en el punto 5 (si existía el registro buscado), y determine cuántas palabras de esa cadena comenzaron con una vocal (minúscula o mayúscula) y contenían al mismo tiempo una secuencia de la forma "k+dígito" (por ejemplo "1k1" o "k9"). Como se dijo, la cadena debe terminar con un punto y las palabras deben separarse entre ellas con un (y solo un) espacio en blanco. La cadena debe ser procesada carácter a carácter, a razón de uno por cada vuelta del ciclo que itere sobre ella.

Criterios generales de evaluación:

- a.) Desarrollo del programa completo, incluyendo los dos módulos pedidos como mínimo, el menú correctamente planteado, funciones bien diseñadas y parametrizadas (cuando sea apropiado), validaciones cuando sean aplicables y estilo de código fuente: **[máximo: 2 puntos (8.3% del puntaje)]**
- b.) Desarrollo correcto de los ítems 1 y 2: **[máximo: 4 puntos (16.6% del puntaje)]**
- c.) Desarrollo correcto del ítem 3: **[máximo: 4 puntos (16.7% del puntaje)]**
- d.) Desarrollo correcto del ítem 4: **[máximo: 4 puntos (16.7 % del puntaje)]**
- e.) Desarrollo correcto del ítem 5: **[máximo: 4 puntos (16.7% del puntaje)]**
- f.) Desarrollo correcto del ítem 6: **[máximo: 2 puntos (8,3 % del puntaje)]**
- g.) Desarrollo correcto del ítem 7: **[máximo: 4 puntos (16.7% del puntaje)]**

Para aprobar el parcial, el alumno debe llegar a un *total acumulado de al menos 60% del puntaje (es decir, alrededor de 14.4 puntos acumulados)*, pero obligatoriamente debe estar desarrollado el programa, funcionando y operativo.

NOTA	PORCENTAJE	CALIFICACIÓN
1		Insuficiente
2		Insuficiente
3		Insuficiente
4		Insuficiente
5		Insuficiente
6	60% a 68%	Aprobado
7	69% a 77%	Bueno
8	78% a 86%	Muy Bueno
9	87% a 95%	Distinguido
10	96% a 100%	Sobresaliente